Exercises : Chapter 3 and 4

Course: Business Intelligence (BI)
Level: 4th Year Artificial Intelligence Engineering

Exercise: Furniture Sales

Part 1: Designing a Data Warehouse

A furniture sales company wants to create a data warehouse. Consider the following information:

- The data warehouse should allow analysis with respect to the furniture sold, customers, and time.
- A piece of furniture is associated with a type (chair, table, display cabinet, etc.), a category (kitchen, living room, bathroom, etc.), and a material (wood, marble, etc.).
- A customer should be associated with a location (city, region, and country).
- The company is interested in the number of sales, revenue, and discounts on completed sales.

Question 1. Design a multidimensional model for the data warehouse based on the requirements above. You must explicitly specify the **grain** of the analysis, the **dimensions**, and the **facts**.

Question 2. Develop a star schema for a data warehouse corresponding to the above requirements.

Question 3. Develop a snowflake schema for a data warehouse corresponding to the above requirements.

Question 4. The company management decides that storing every single transaction is too expensive. They now only want to analyze the total revenue per Month and per Furniture Category. Describe how this change in **granularity** affects the Fact Table and the Dimensions.

Solution

Question 1: Multidimensional Model (Kimball's 4-Step Process)

Following the standard dimensional modeling methodology:

Step 1: Select the Business Process
The core business process being modeled is **Furniture Sales** (Transactions).

Step 2: Declare the Grain
The grain is defined as **one row per individual completed sale** (atomic transaction line item).

Step 3: Identify the Dimensions (Context)
The analysis will be performed with respect to the following dimensions:

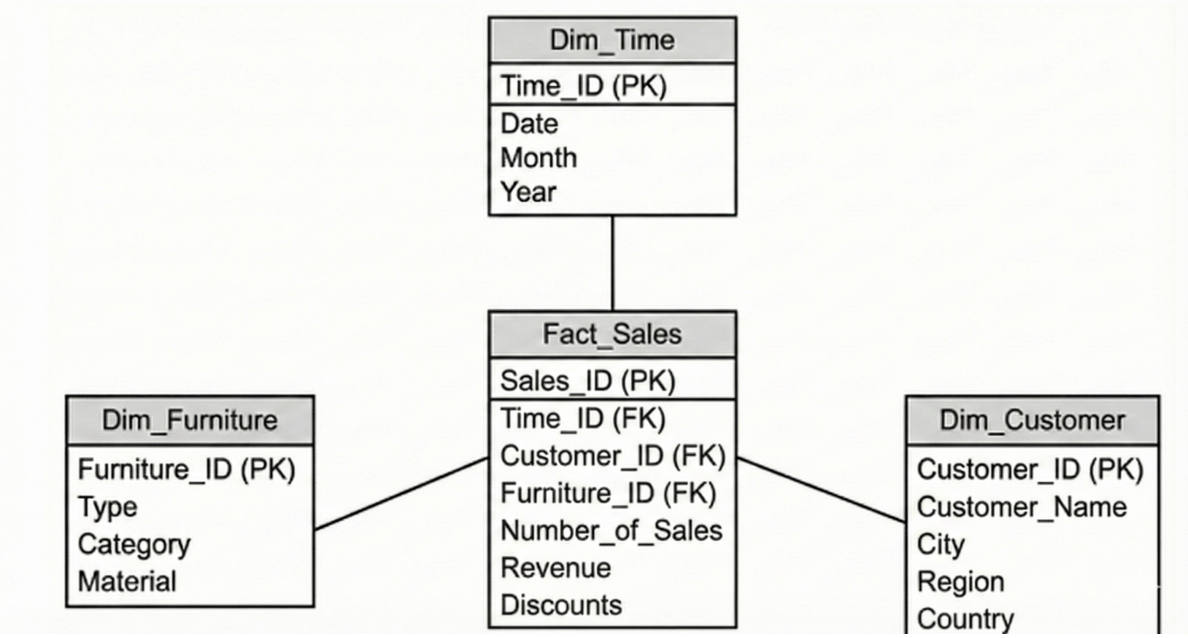
- **Furniture Dimension:** Attributes include Type, Category, and Material.
- **Customer Dimension:** Attributes include Customer name, City, Region, and Country.
- **Time Dimension:** Attributes include Date, Month, and Year.

Step 4: Identify the Facts
The numerical measures to be stored in the Fact Table are:

- Number of sales (Quantity)
- Revenue
- Discounts

Question 2: Star Schema

In the Star Schema, the **Dimensions are not normalized**. We keep all attributes in a single table for each dimension.



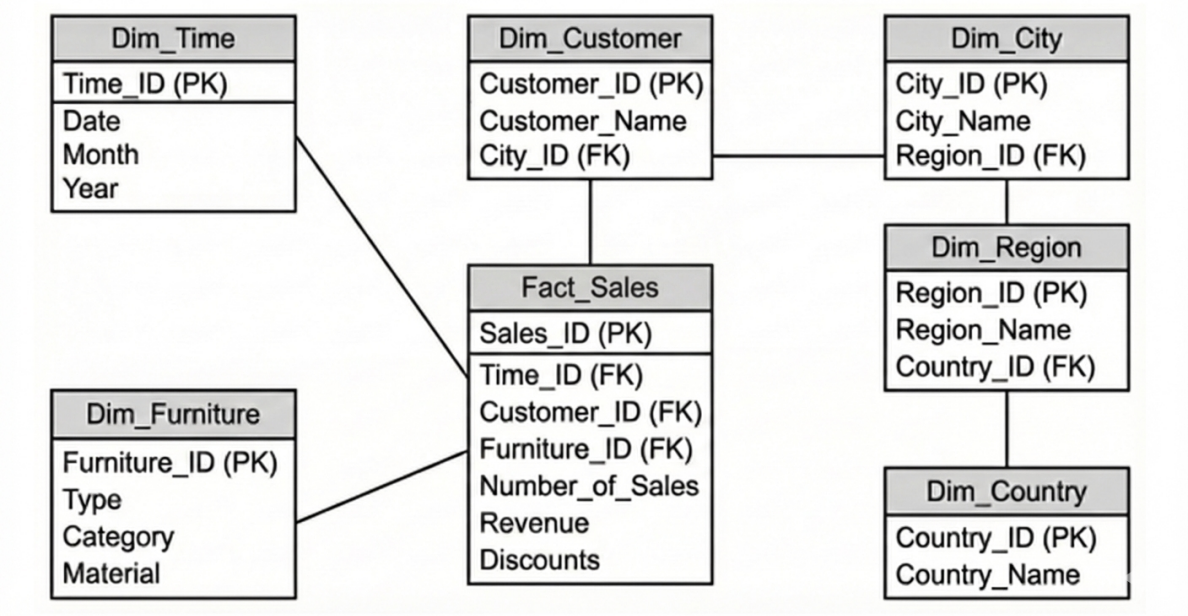
Question 3: Snowflake Schema

In the Snowflake Schema, we **normalize the hierarchies**. The **Customer Dimension** is split because it has a hierarchy (City → Region → Country).

Question 4

Changing to a coarser granularity (Aggregation) involves:

1. **Fact Table:** The grain becomes "One row per Category per Month". The Number_of_Sales becomes a sum of items, not a count of transactions, or the attribute can be removed.
2. **Time Dimension:** We no longer need the Day attribute; the lowest level is now Month.
3. **Furniture Dimension:** We no longer need specific Type or individual item IDs; the data is rolled up to Category.
4. **Customer Dimension:** This dimension might be removed entirely (dropped).



Part 2: Data Analysis and KPI Calculation

Assuming the Data Warehouse is populated with sales data, the management wants to build a Dashboard to monitor performance.

Based on the Star Schema designed in Question 2 Part 1, write the SQL queries to calculate the following KPIs:

1. **Category Performance:** Calculate the Total Revenue generated by each *Furniture Category* in 2024.
2. **Customer Ranking:** Identify the top 3 customers in each region based on their Total Spending. *(Hint: Use the `WITH` clause to calculate total spending per customer first, then rank the results).*
3. **Trend Analysis:** Calculate the Monthly Sales Growth (Difference vs Previous Month) for the year 2024. *(Hint: Use `WITH` to aggregate sales by month first, then compare with the previous month).*
4. **Cumulative Progress:** Calculate the Year-to-Date (YTD) Revenue for the "Kitchen" category to show how revenue accumulates day by day in 2024.
5. **Price Comparison:** Display the revenue of each individual sale alongside the **Average Revenue** of its Category (for comparison).

Important Instructions:

- For **queries (2 to 5)**, you are **required** to use **Window Functions** (See Chapter 4).
- For complex queries (like Question 2 and 3), use the `WITH` clause to create a temporary "helper table" to simplify your code before applying the Window Function.

```sql
WITH Temporary_Table_Name AS (
    SELECT ... -- Calculate intermediate results (e.g., Sums)
    FROM ...
    GROUP BY ...
)
SELECT ...
FROM Temporary_Table_Name
WHERE ...; -- Perform final ranking or analysis
```

Solution

Note: All queries are written based on the **Star Schema** designed in Question 2.

1. Category Performance (Basic Aggregation)

Goal: Calculate the Total Revenue generated by each Furniture Category in 2024.

```sql
SELECT
    DF.Category,
    SUM(FS.Revenue) AS Total_Revenue
FROM Fact_Sales FS
JOIN Dim_Furniture DF ON FS.Furniture_ID = DF.Furniture_ID
JOIN Dim_Time DT ON FS.Time_ID = DT.Time_ID
WHERE DT.Year = 2024
GROUP BY DF.Category;
```

2. Customer Ranking (Ranking Window Function)

Goal: Identify the top 3 customers in each region based on their Total Spending.

Explanation: We use a Common Table Expression (CTE) first to calculate the total spending per customer, and then we rank them in the main query.

```sql
WITH Customer_Spending_table AS (
    SELECT
        DC.Region,
        DC.Customer_Name,
        SUM(FS.Revenue) AS Total_Spent
    FROM Fact_Sales FS
    JOIN Dim_Customer DC ON FS.Customer_ID = DC.Customer_ID
    GROUP BY DC.Region, DC.Customer_Name
)
SELECT * FROM (
    SELECT
        Region,
        Customer_Name,
        Total_Spent,
        RANK() OVER (PARTITION BY Region ORDER BY Total_Spent DESC) as Rank_Num
    FROM Customer_Spending_table
)
WHERE Rank_Num <= 3;
```

3. Trend Analysis (LAG Window Function)

Goal: Calculate the Monthly Sales Growth (Current vs Previous Month) for 2024.

```sql
WITH Monthly_table AS (
    SELECT
        DT.Month,
        SUM(FS.Revenue) AS Current_Sales
    FROM Fact_Sales FS
    JOIN Dim_Time DT ON FS.Time_ID = DT.Time_ID
    WHERE DT.Year = 2024
    GROUP BY DT.Month
)
SELECT
    Month,
    Current_Sales,
    LAG(Current_Sales, 1) OVER (ORDER BY Month) AS Previous_Sales,
    (Current_Sales - LAG(Current_Sales, 1) OVER (ORDER BY Month)) AS Growth_Diff
FROM Monthly_table;
```

4. Cumulative Progress (Running Total)

Goal: Calculate the Year-to-Date (YTD) Revenue for the "Kitchen" category in 2O24.

Explanation: We sum the daily sales first, then apply the window function to create a running total accumulating over time.

```
SELECT
    DT.Date,
    SUM(FS.Revenue) AS Daily_Sales,
    SUM(SUM(FS.Revenue)) OVER (PARTITION BY DT.Year ORDER BY DT.Date) AS
   YTD_Revenue
FROM Fact_Sales FS
JOIN Dim_Time DT ON FS.Time_ID = DT.Time_ID
JOIN Dim_Furniture DF ON FS.Furniture_ID = DF.Furniture_ID


WHERE DF.Category = 'Kitchen'
GROUP BY DT.Date;
```

5. Price Comparison (Partitioning without Grouping)

Goal: Display the revenue of each individual sale alongside the Average Revenue of its Category.

Explanation: Here we do NOT use GROUP BY because we want to keep the individual transaction rows while showing the aggregate average next to them.

```
SELECT
    DF.Category,
    FS.Revenue AS Individual_Sale_Price,
    AVG(FS.Revenue) OVER (PARTITION BY DF.Category) AS Avg_Category_Price
FROM Fact_Sales FS
JOIN Dim_Furniture DF ON FS.Furniture_ID = DF.Furniture_ID;
```